Lesson 11 Teacher's Script

Core Mechanics V: Wave After Wave

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week your game became genuinely replayable - a real freeze, a real restart. Quick show of hands - who tested a full win-then-restart AND a full lose-then-restart?" [Pause, call on 1-2 volunteers]

"Two things from last week I want you to carry into today. First, game state: when you restart, EVERYTHING that changed has to be put back, including anything extra you added. Second, the bug. Some of you found a round that simply never ends - you destroyed some enemies, let the others escape, and nothing happened: no win, no loss, and R did nothing. Let's give it a name: the Stalemate. Hold that thought. It's still in your game, on purpose, and today it's about to get more interesting." [Do NOT explain the fix. If a student asks, say: 'What has to be true for a round to be over?']

"Today's a big one - by the end of class, your game reaches its second major milestone of the year. Clearing one wave of enemies is about to become clearing several, in a row, before you actually win."

[0:05-0:20] One More Question: Is This the Last Wave?

"Since Lesson 6, your win-check has asked exactly one question: are all the enemies gone? Today, that question grows a second part: gone, AND is this the last wave? If both are true, you really win. If enemies are gone but more waves remain, the round just... continues."

"This is the first time this course asks you to nest one decision inside another. It's a small change in code, but it's a genuinely new shape of thinking - a decision whose answer depends on a second, smaller decision."

[Discussion question] "Can anyone think of a decision in real life that works this way - where the answer to one question depends on a second question nested inside it?" (Examples to draw out if needed: "can I go to the party" depends on "is my homework done," which itself might depend on "do I understand the assignment.")

[Timing checkpoint: wrap by 0:15]

[0:15-0:20] The Whole Decision, One Picture

[Show the next slide: the Activity Diagram] "Here's the whole decision in one picture. An enemy is destroyed: enemies_remaining goes down by one. Is it zero? If not, the wave simply

continues. If it is, the wave is cleared - and now the second question: is this the last wave? Yes: the real win, and R restarts as before. No: the next wave. wave goes up by one, enemies_remaining goes back to five, and we call spawn_enemies again. Notice score, lives and game_active are untouched. This is the same picture in every engine, whatever the code looks like."

[Timing checkpoint: wrap by 0:20]

[0:20-0:40] The Same Spawn Code, Called a Third Way

"Here's the good news: you're not writing new enemy-spawning code today. You've had it since Lesson 5, and Lesson 10 already made it callable a second way, for restarts. Today it gets a THIRD way to run: every time a wave is cleared, if more waves remain."

[Demo cue: switch to Unity, open last week's project and GameManager.cs]

"Watch. SpawnEnemies() on EnemySpawner already has two callers: Start() and ResetGame(). Today I add three static fields to GameManager: wave, totalWaves and nextWaveRequested. Then I turn RegisterKill()'s win check into an if/else: if wave is at least totalWaves, hasWon and gameActive, exactly as before. Otherwise wave++, enemiesRemaining back to 5, and nextWaveRequested = true." [Build the if/else live]

"Why a flag, and not just spawner.SpawnEnemies() right there? Try it." [Type spawner.SpawnEnemies() inside RegisterKill() and let the compiler complain] "RegisterKill() is static - it belongs to the GameManager class, not to the one GameManager object in the scene, and the Spawner slot we filled in last week belongs to that object. A static method can't see it. So RegisterKill() leaves a note, and GameManager's own Update(), which does belong to the object, checks the note, clears it and calls spawner.SpawnEnemies()." [Add the nextWaveRequested check to Update()] "That's the real cost of static members. We'll come back to it in Semester 2."

[Demo cue: add public Text waveText to HUD.cs, one line in Update(), drag WaveText onto the slot in the Inspector, press Play and clear wave 1 into wave 2]

"Same enemy-creation method, zero changes to it. Three callers in: EnemySpawner's Start(), every restart through ResetGame(), and now every wave clear through GameManager's Update(). That's the whole idea."

[Timing checkpoint: wrap by 0:35]

[0:35-0:40] New State, Same Habit

[Show the next slide: New State, Same Habit] "Last week's habit applies the moment you add anything new. wave and totalWaves are new game state, and they live on the GameManager, just like score and lives, because more than one object cares about them. So: does the restart put wave back to 1? I'll check it right now." [Demo cue: in the Unity demo, reach wave 3, lose, press R, and point at the HUD: it must say Wave: 1. If you forgot ResetGame()'s wave = 1, you will see Wave: 3 - leave that on screen for a moment before fixing it.]

"And if you add a wave message today, an effect, a sound, anything: ask the same two questions. What does it change when the round ends or the wave changes? And what puts it back?"

[Timing checkpoint: wrap by 0:40]

[0:40-0:50] Milestone Moment: What "Level 2 Complete" Actually Means

"Take a second to think about everything your game can do now: move, shoot, face a wave of enemies, track a score, survive with limited lives, restart cleanly - and now, face MULTIPLE waves before really winning. That's not a small list. That's the Level 2 Complete milestone from your capstone's design document."

"Back in Lesson 6, Level 1 MVP meant 'a complete, playable loop, start to finish.' Level 2 Complete means something more: a game with actual depth - failure that matters, progress that builds, and a real sense of escalation as more waves arrive. This is the kind of thing that starts to feel like an actual arcade game, not just a working prototype."

"From here through Lesson 15, everything is about making this same loop feel good - sound, animation, difficulty, and polish. The mechanics are done. What's left is making them sing."

[Timing checkpoint: wrap by 0:50]

[0:50-0:55] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one, and no new assets needed. Today's goal: clearing a wave spawns a new one, up to 3, and only clearing wave 3 is a real win."

[Hand out engine-specific handouts]

"Test a FULL 3-wave playthrough today, not just wave 1 - and separately, make sure losing your last life mid-wave still works correctly, whichever wave you're on."

"One more thing. The Stalemate is still in there, and now it can freeze the game at any of the three waves. If it starts to annoy you, that's your Level 3. You already know how to find it."

"Questions before we start lab? ... Let's build your second milestone."

Lab Time Notes

- The single most common bug this week: forgetting the wave-count check entirely, so clearing wave 1 immediately shows the win message. Test this explicitly - the win should NOT appear until the final wave.
- Second most common: the wave-advance branch accidentally touching score or lives - only enemies_remaining and the wave counter should change between waves.
- Game-state drift: wave not reset on restart. Have students test restarting from mid-wave-2 or mid-wave-3, not just from a fresh game - restart should always return to wave 1. Point them back to the 'Check Your Game State' list in the Handout.
- The Stalemate: students will start reporting 'my wave never ends.' Confirm it is real, name it, do not hand over the fix, and send them back to their Lesson 10 Level 3 notes. Students who already fixed it should check that the fix works at every wave. Each Key's Level 3 section has the worked fix.
- Engine notes: Godot spawns the next wave with call_deferred (a wave clears mid-collision); Unity requests the next wave with a flag that GameManager's Update() acts on; Scratch broadcasts [spawn enemies]; GameMaker calls spawn_enemies(), with a Function Call block in Visual. A doubled wave in Scratch usually means a spawn script is still on [start game].
- Circulate by engine cluster as usual. At 1:50, call time and do a real celebration share-out - this is the Level 2 Complete milestone. Consider having 2-3 students demo a full 3-

wave playthrough from start to finish, the same energy as Lesson 6's milestone share-out.